

USER ACCEPTANCE TESTS (UATs)

StokPal Group Management & Invite System

UAT 1 — Create Group Successfully

User Story

As a registered user, I want to create a stokvel group so that I can manage members and contributions.

Acceptance Criteria

- User can create a group
- User becomes group admin automatically
- Membership record is created
- User is redirected to dashboard
- Group appears on dashboard

Given / When / Then

Given

The user is authenticated and logged into the application.

When

The user completes the Create Group form and clicks “Create Group”.

Then

- A new group document is created in Firestore
- The creator is added to the group members subcollection with role “admin”
- A membership record is created
- A success message is displayed
- The user is redirected back to the dashboard
- The newly created group appears on the dashboard

Expected Result

Group creation completes successfully without Firebase permission errors.

UAT 2 — Send Invite to User

User Story

As a group admin, I want to invite users to my stokvel group so that they can join the group.

Acceptance Criteria

- Admin can send invite
- Invite is stored in Firestore
- Invite status defaults to “pending”
- Invite expiration date is created

Given / When / Then

Given

The authenticated user is an admin of a group.

When

The admin enters an email address and clicks “Send Invite”.

Then

- An invite document is created in Firestore
- The invite stores:
 - email
 - groupId
 - invitedBy
 - createdAt
 - expiresAt
 - status = “pending”

Expected Result

The invite is stored successfully and becomes available to the invited user.

UAT 3 — Accept Invite

User Story

As an invited user, I want to accept a stokvel invite so that I can join the group.

Acceptance Criteria

- User can view pending invites
- User can accept invite
- Membership is created
- User becomes a group member

- Invite status changes to “accepted”

Given / When / Then**Given**

A pending invite exists for the authenticated user’s email.

When

The user clicks the “Accept” button.

Then

- The user is added to:
groups/{groupId}/members
- A membership record is created
- The invite status becomes “accepted”
- The group appears on the dashboard

Expected Result

The user joins the group successfully and can access group functionality.

UAT 4 — Decline Invite**User Story**

As an invited user, I want to decline an invite so that I can reject unwanted invitations.

Acceptance Criteria

- User can decline invite
- Invite status updates correctly
- Invite disappears from pending list

Given / When / Then**Given**

A pending invite exists for the authenticated user.

When

The user clicks the “Decline” button.

Then

- The invite status updates to “declined”
- The invite no longer appears in pending invites

Expected Result

The invite is successfully declined.

UAT 5 — Resend Expired Invite

User Story

As a group admin, I want to resend expired invites so that users can still join the group.

Acceptance Criteria

- Admin can resend expired invites
- Expiration date refreshes
- Invite status resets to pending

Given / When / Then

Given

An invite has expired.

When

The admin clicks “Resend Invite”.

Then

- expiresAt updates
- status becomes “pending”
- invited user can accept again

Expected Result

The invite becomes active again.

UAT 6 — Schedule Meeting

User Story

As an admin or treasurer, I want to schedule meetings so that members know upcoming meeting dates.

Acceptance Criteria

- Admin/treasurer can create meetings
- Meetings appear in dashboard
- Meeting details save correctly

Given / When / Then

Given

The user is either:

- admin
- treasurer

of a group.

When

The user submits the Schedule Meeting form.

Then

- A meeting document is created in Firestore
- The meeting displays in the meetings section
- Group members can view the meeting

Expected Result

Meetings are successfully scheduled and displayed.

UAT 7 — Role-Based Access Control

User Story

As a system administrator, I want users to have role-based permissions so that only authorized users can perform sensitive actions.

Acceptance Criteria

- Only admins can:
 - delete groups
 - assign admins
 - assign treasurers
 - edit groups
- Members cannot access admin functionality

Given / When / Then

Given

A user has role “member”.

When

The user attempts to access admin-only functionality.

Then

The action is denied.

Expected Result

Permissions are enforced correctly using Firebase rules and frontend checks.

UAT 8 — Dashboard Group Rendering

User Story

As a user, I want my groups to appear on the dashboard so that I can access group information.

Acceptance Criteria

- Dashboard loads memberships
- Group cards display correctly
- Newly joined groups appear automatically

Given / When / Then

Given

The user belongs to one or more groups.

When

The dashboard loads.

Then

- Memberships are queried
- Related groups are fetched
- Group cards display on screen

Expected Result

Dashboard accurately reflects user memberships.

UAT 9 — Firebase Rules Enforcement

User Story

As a developer, I want Firebase rules to secure the database so that unauthorized actions are prevented.

Acceptance Criteria

- Unauthorized users cannot:
 - edit groups
 - modify payouts

- delete meetings
 - assign roles
- Authorized users can perform valid actions

Given / When / Then

Given

A user lacks required permissions.

When

The user attempts a restricted action.

Then

Firestore denies the request.

Expected Result

Database security rules prevent unauthorized access.

UAT 10 — Role Standardization Fix

User Story

As a developer, I want role naming to be standardized so that permissions behave consistently across the system.

Acceptance Criteria

- All roles use lowercase:
 - admin
 - treasurer
 - member
- Meetings and invites function correctly

Given / When / Then

Given

The application uses role checks.

When

A role comparison occurs.

Then

The comparison succeeds consistently.

Expected Result

Role-based features function without capitalization mismatches.